

EXPERIMENT - 30

1. Problem Statement

Write a C program to perform the following menu-driven operations on a given array:

- Sort the elements using the Bubble Sort algorithm and perform a Binary search.
- Perform Insertion and Deletion in a given valid position.
- Rotate the array elements to the right by k positions.
- Exit.

2. Algorithm

Main Program

Step 1: START

Step 2: Read n and then read n elements into the array a. This is the given array, and every operation works on it.

Step 3: Display the menu and read choice.

Step 4: Using a switch statement on choice:

 If choice is 1, call bubbleSortAndSearch().

 If choice is 2, call insertAndDelete() and store the size it returns back in n.

 If choice is 3, call rotateRight().

 If choice is 4, display "Exiting the program."

Step 5: Repeat from Step 3 while choice is not equal to 4.

Step 6: STOP

Function bubbleSortAndSearch()

Step 1: Repeat for i = 0 to n - 2:

 Repeat for j = 0 to n - 2 - i: if a[j] is greater than a[j + 1], swap them.

Step 2: Display the sorted array.

Step 3: Read the element to be searched, key. Set low = 0, high = n - 1 and found = 0.

Step 4: Repeat while low is less than or equal to high:

 Set mid = (low + high) / 2.

 If a[mid] is equal to key, display the position mid + 1, set found = 1 and stop the loop.

 Else if a[mid] is less than key, set low = mid + 1, otherwise set high = mid - 1.

Step 5: If found is still 0, display that the element is not present.

Step 6: Return to the calling function.

Function insertAndDelete()

Step 1: Read pos. While pos is less than 1 or greater than n + 1, display a message and read pos again.

Step 2: Read the element to be inserted, value.

Step 3: Repeat for i = n down to pos: set a[i] = a[i - 1].

Step 4: Set $a[\text{pos} - 1] = \text{value}$, increase n by 1 and display the array.

Step 5: Read pos again. While pos is less than 1 or greater than n , display a message and read pos again.

Step 6: Display the element $a[\text{pos} - 1]$ that is being removed.

Step 7: Repeat for $i = \text{pos} - 1$ to $n - 2$: set $a[i] = a[i + 1]$.

Step 8: Decrease n by 1, display the array and return n .

Function rotateRight()

Step 1: Read k and set $k = k \% n$.

Step 2: Repeat for $i = 0$ to $n - 1$: set $b[(i + k) \% n] = a[i]$.

Step 3: Repeat for $i = 0$ to $n - 1$: set $a[i] = b[i]$.

Step 4: Display the array.

Step 5: Return to the calling function.

3. Test Case Table

#	Operation	Input	Expected Output	Actual Output	Result
1	1. Sort + search	40 10 30 20 50, key = 30	Sorted 10 20 30 40 50, found at position 3	Same	PASS
2	1. Sort + search	key = 99	99 is not present in the array	Same	PASS
3	2. Insertion	99 at position 2	10 99 20 30 40 50	Same	PASS
4	2. Deletion	position 4	30 removed, array 10 99 20 40 50	Same	PASS
5	2. Invalid position	0, then 9, then 2	The message is shown twice and 2 is accepted	Shown twice	PASS
6	3. Rotate	$k = 2$ on 7 99 20 40 50	40 50 7 99 20	Same	PASS
7	3. Rotate	$k = 7$ on 40 50 7 99 20	Same as $k = 2$, giving 99 20 40 50 7	Same	PASS
8	4. Exit	choice = 4	Exiting the program, and the loop ends	Program ended	PASS

4. Source Code

arrayops30.c

```
/* Experiment 30. Menu driven program that works on one given array.
   The array is read once at the start, and every operation in the menu
   works on that same array, so a change made by one option is still
   there when the next option runs. */

#include <stdio.h>
#define MAX 100

void display(int a[], int n);
void bubbleSortAndSearch(int a[], int n);
int insertAndDelete(int a[], int n);
void rotateRight(int a[], int n);

int main(void)
{
    int a[MAX], n, i, choice;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    do {
        printf("\nMENU\n");
        printf("1. Sort using Bubble Sort and perform Binary search\n");
        printf("2. Insertion and Deletion at a given valid position\n");
        printf("3. Rotate the array to the right by k positions\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: bubbleSortAndSearch(a, n); break;
            case 2: n = insertAndDelete(a, n); break;
            case 3: rotateRight(a, n); break;
            case 4: printf("Exiting the program.\n"); break;
        }
    } while (choice != 4);

    return 0;
}

/* Prints the array on one line. */
void display(int a[], int n)
{
    int i;

    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

/* Bubble sort puts the largest remaining element at the end in every
   pass, so after the pass number p the last p places are already
   settled and the inner loop can stop that much earlier. Binary search
   then works because the array is sorted. */
void bubbleSortAndSearch(int a[], int n)
{
    int i, j, temp, key, low, high, mid, found = 0;

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
            }
        }
    }
}
```

```

        a[j + 1] = temp;
    }
}

printf("Sorted array: ");
display(a, n);

printf("Enter the element to be searched: ");
scanf("%d", &key);

low = 0;
high = n - 1;
while (low <= high) {
    mid = (low + high) / 2;
    if (a[mid] == key) {
        printf("%d found at position %d\n", key, mid + 1);
        found = 1;
        break;
    }
    else if (a[mid] < key)
        low = mid + 1;
    else
        high = mid - 1;
}

if (found == 0)
    printf("%d is not present in the array\n", key);
}

/* Insertion makes room by shifting the elements from the end backwards,
   and deletion closes the gap by shifting them forward. The new size is
   returned so that main can keep using it. */
int insertAndDelete(int a[], int n)
{
    int pos, value, i;

    printf("Enter the position for insertion (1 to %d): ", n + 1);
    scanf("%d", &pos);
    while (pos < 1 || pos > n + 1) {
        printf("Invalid position. Enter again (1 to %d): ", n + 1);
        scanf("%d", &pos);
    }

    printf("Enter the element to be inserted: ");
    scanf("%d", &value);

    for (i = n; i >= pos; i--)
        a[i] = a[i - 1];
    a[pos - 1] = value;
    n = n + 1;

    printf("Array after insertion: ");
    display(a, n);

    printf("Enter the position for deletion (1 to %d): ", n);
    scanf("%d", &pos);
    while (pos < 1 || pos > n) {
        printf("Invalid position. Enter again (1 to %d): ", n);
        scanf("%d", &pos);
    }

    printf("Element deleted: %d\n", a[pos - 1]);
    for (i = pos - 1; i < n - 1; i++)
        a[i] = a[i + 1];
    n = n - 1;

    printf("Array after deletion: ");
    display(a, n);

    return n;
}

/* Rotating to the right by k means the element at place i goes to place

```

```

    (i + k) % n. A copy is used so that no element is overwritten before
    it has been moved. If k is larger than n, k % n gives the same
    result with less work. */
void rotateRight(int a[], int n)
{
    int b[MAX], k, i;

    printf("Enter the value of k: ");
    scanf("%d", &k);

    k = k % n;

    for (i = 0; i < n; i++)
        b[(i + k) % n] = a[i];

    for (i = 0; i < n; i++)
        a[i] = b[i];

    printf("Array after rotating right by %d: ", k);
    display(a, n);
}

```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra arrayops30.c -o arrayops30` so that even the small warnings would show up, and it compiled clean in the first attempt itself. All the screenshots below are from one single run, so the array carries its changes from one option to the next.

Bubble Sort and Binary Search

```

adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra arrayops30.c -o arrayops30
adilshakil@ADILs-MacBook-Air MyAssignment % ./arrayops30
Enter the number of elements: 5
Enter 5 elements: 40 10 30 20 50

MENU
1. Sort using Bubble Sort and perform Binary search
2. Insertion and Deletion at a given valid position
3. Rotate the array to the right by k positions
4. Exit
Enter your choice: 1
Sorted array: 10 20 30 40 50
Enter the element to be searched: 30
30 found at position 3

MENU
1. Sort using Bubble Sort and perform Binary search
2. Insertion and Deletion at a given valid position
3. Rotate the array to the right by k positions
4. Exit
Enter your choice: 1
Sorted array: 10 20 30 40 50
Enter the element to be searched: 99
99 is not present in the array

```

Insertion and Deletion, and an Invalid Position

```
adilshakil — -zsh — 120x33

MENU
1. Sort using Bubble Sort and perform Binary search
2. Insertion and Deletion at a given valid position
3. Rotate the array to the right by k positions
4. Exit
Enter your choice: 2
Enter the position for insertion (1 to 6): 2
Enter the element to be inserted: 99
Array after insertion: 10 99 20 30 40 50
Enter the position for deletion (1 to 6): 4
Element deleted: 30
Array after deletion: 10 99 20 40 50

MENU
1. Sort using Bubble Sort and perform Binary search
2. Insertion and Deletion at a given valid position
3. Rotate the array to the right by k positions
4. Exit
Enter your choice: 2
Enter the position for insertion (1 to 6): 0
Invalid position. Enter again (1 to 6): 9
Invalid position. Enter again (1 to 6): 2
Enter the element to be inserted: 7
Array after insertion: 10 7 99 20 40 50
Enter the position for deletion (1 to 6): 1
Element deleted: 10
Array after deletion: 7 99 20 40 50
█
```

Rotation to the Right, and Exit

```
adilshakil — -zsh — 120x33

MENU
1. Sort using Bubble Sort and perform Binary search
2. Insertion and Deletion at a given valid position
3. Rotate the array to the right by k positions
4. Exit
Enter your choice: 3
Enter the value of k: 2
Array after rotating right by 2: 40 50 7 99 20

MENU
1. Sort using Bubble Sort and perform Binary search
2. Insertion and Deletion at a given valid position
3. Rotate the array to the right by k positions
4. Exit
Enter your choice: 3
Enter the value of k: 7
Array after rotating right by 2: 99 20 40 50 7

MENU
1. Sort using Bubble Sort and perform Binary search
2. Insertion and Deletion at a given valid position
3. Rotate the array to the right by k positions
4. Exit
Enter your choice: 4
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
█
```

All eight test cases came out right. Binary search only works on a sorted array, which is why the question puts the sorting and the searching in the same option. For rotation I used a second array to hold the result, because writing straight back into the same array overwrites an element before it has been moved. Giving $k = 7$ on five elements produced the same answer as $k = 2$, which shows the $k \% n$ step is doing its job.